QALA

Version 1 — Minimum Viable Product

Product Proposal

"See and control everything you're building — as one system."

Solution Orchestration Layer | Connect. Structure. Operate.

Classification: Internal — Product Document
April 2026 | v1.0

1. Executive Summary

Qala is a solution orchestration platform — a unified control layer that sits above existing development, deployment, and distribution toolchains. It connects them, maps everything a team is building into a single structured model, and gives builders real-time visibility and operational control over their entire system.

This document defines the scope, rationale, and delivery plan for Qala Version 1 — the Minimum Viable Product (MVP). V1 is a deliberately narrow, high-value entry wedge designed to validate the core thesis, generate immediate user value, and establish the platform foundation for future growth.

> **V1 Core Thesis**
> There is no unified system today that represents everything a team is building as a coherent, executable structure.
> Qala fills that gap — without requiring users to replace their existing tools.
> V1 delivers this thesis in 60–90 seconds: connect your tools → see your entire system instantly.

V1 focuses exclusively on four core capabilities:

- **System Graph:** Auto-generated visual map of all repositories, services, dependencies, and deployment environments — produced immediately from tool connections with zero manual input.
- **Solution Registry:** A structured, searchable catalog of all solutions being built, with lightweight metadata and lifecycle state tracking.
- **Tool Integration Layer:** Adapters for GitHub, Vercel, and AWS that ingest and normalize external system state into the unified Qala model.
- **Tile Dashboard:** A real-time, composable decision surface showing system health, deployment status, alerts, and key metrics through stateful, interactive tiles.

Everything else in the broader Qala platform vision — the DSL, full governance, supply chain management, IP management, kogi/ume integration, advanced analytics — is explicitly deferred beyond V1.

2. Vision & Ecosystem Context

2.1 Platform Ecosystem: kogi · ume · qala

Qala is one layer of a three-platform ecosystem designed for systematic, deterministic outcomes across the full cycle of organizational value creation.

Platform	Domain	Function	V1 Role
kogi	Portfolio / Input Management	Portfolio state estimation and optimization. Manages inputs — requirements, signals, capital.	Future — post-V1
ume	Organization / Transformation	Organization state estimation and optimization. Manages transformations and work.	Future — post-V1
qala	Solution / Output Management	Solution state estimation and optimization. Manages outputs — solutions, artifacts, deployments.	THIS PROPOSAL

Together, the three platforms form a closed loop:

> ***Define (Qala) → Execute (Ume) → Measure (Kogi) → Optimize (Kogi) → Update System (Qala)***

V1 establishes the Define layer — the foundation that makes the entire loop possible.

2.2 What Qala Is — and Is Not

Qala is NOT a replacement for existing tools. It does not compete with GitHub, Vercel, AWS, Terraform, Linear, or Datadog. Each of these continues to serve its purpose.

> ***"If your tools are instruments, qala is the conductor — not a replacement for the instruments."***

Existing Tool	What It Does	Qala's Relationship
GitHub	Manages source code	Integrates — repos → solution components
Vercel / AWS	Runs deployments	Integrates — deployments → environment state
GitHub Actions / CI	Executes pipelines	Integrates — pipeline runs → execution signals
Datadog / observability	Monitors running systems	Future integration — feeds observatory
Linear / Jira	Manages tasks	Future integration — feeds work system
Backstage	Documents systems (static)	Qala is a live, executable, stateful version of this

2.3 The Core Mental Model

The foundational data model underpinning qala is the Distributed Solution Spreadsheet — a living, versioned, multi-dimensional record of every solution and all of its constituent parts, relationships, configurations, states, and lifecycle events.

Spreadsheet Concept	Qala Equivalent	Description
Row	Solution Record	A single Solution entry with all metadata, type, maturity, and version
Column	Solution Field	A named property: ID, name, version, type, maturity, owner, SDE reference
Sheet	Portfolio / Factory	A scoped collection of Solution records
Formula / Function	Solution Operation	snapshot(), rollback(), promote_maturity(), assemble()
Workbook	Solution Factory	The full collection of sheets managed by a factory
Workspace	Operational Layer	Where users directly read and write to the Solution Spreadsheet

3. Problem Statement

Modern builders — especially founders, AI developers, and early-stage technical teams — operate in deeply fragmented environments. Their systems are real, but invisible as systems.

The Fragmentation Reality	The Consequences
• Code lives in repositories (GitHub) • Infrastructure is defined separately (Terraform, AWS) • Deployments occur across multiple platforms • Tasks and planning exist in disconnected tools • Dependencies and system relationships are implicit	• No system-level visibility • High cognitive load across all team members • Inefficient cross-tool coordination • Difficulty scaling system complexity • No unified model for execution and optimization

> ***The market gap: No platform today provides a unified, executable model of an entire system. Teams know what they're building. They don't know the state of what they're building.***

4. Target Users — V1

V1 targets builders who personally feel system fragmentation as a daily operational pain. These are not enterprise buyers — they are technical owners who will self-serve, integrate immediately, and become the internal champions for broader adoption.

Persona	Profile	Primary Pain	V1 Value Unlocked
System Builder Founders	Indie founders building multiple interconnected platforms (e.g., kogi + ume + qala)	No single view of all systems; dependency hell; unclear deployment states	Instant graph of all systems and their relationships; deployment status in one place
AI Systems Builders	Developers managing agent pipelines, APIs, and workflow systems	No structure for agent system topology; unclear which services feed which	Visual map of AI pipeline architecture; dependency tracking; environment state
Pre-PMF Technical Teams	Small teams (2–8 people) with rapidly evolving stacks	Context switching between tools; onboarding new members is painful; system drift	Shared source of truth for what exists and what state it is in
Platform / Infra Teams	Internal platform engineers managing developer tooling at growing orgs	Service catalog is stale; no executable system view; governance is manual	Live catalog with real state; dependency graphs; future governance foundation

5. V1 MVP Scope

The V1 scope is defined by one principle: deliver the core thesis — connect tools, see your system instantly — with the minimum surface area needed to make it real, useful, and trustworthy.

5.1 In Scope — V1 Delivers

Feature Area	V1 MVP Scope	Status
Onboarding & Workspace Setup	Account creation, personal factory initialization, guided connect-tools flow	In Scope
Tool Integration — GitHub	OAuth connection; repo scanning; infer components, dependencies from package.json / config files; webhook ingestion	In Scope
Tool Integration — Vercel	OAuth connection; deployment ingestion; environment mapping; status sync	In Scope
Tool Integration — AWS	API key / IAM connection; service discovery; deployment state ingestion	In Scope
System Graph	Auto-generated visual dependency graph from integrations; nodes = services/components, edges = dependencies, layers = environments; interactive; real-time	In Scope
Solution Registry (lite)	Create, read, update solutions; basic metadata (name, type, version, maturity, owner); solution catalog with search and filter	In Scope
Solution Maturity Lifecycle (lite)	SANDBOX → DEV → TEST tracking; manual promotion; basic gate awareness (no automated enforcement)	In Scope
Solution Components (lite)	Link repos/services to solutions as components; manually add or auto-discover from integrations	In Scope
Deployment State Tracking	Per-solution, per-environment deployment status; last deploy time; health status; failure detection	In Scope
Dashboard — Tile System (basic)	Tile object model; 5 tile types (status, metric, alert, relationship, action); manual grouping and layout; basic relevance sorting; real-time updates via WebSockets	In Scope
Solution Workspace (lite)	Basic solution detail page; editable metadata; component list; linked deployments; notes field	In Scope

Solution Book (lite)	Charter fields (vision, mission, brief, goals, roadmap); notes; basic document attachment	In Scope
First-Run Experience	Connect → see system in 60–90 seconds; zero manual input required; guided highlights of first insights	In Scope
Notifications & Alerts	Email and in-app alerts for deployment failures, status changes, and system anomalies	In Scope
User Auth & RBAC (basic)	Auth (email + OAuth); workspace-level roles: Owner, Member, Viewer	In Scope

5.2 Out of Scope — Explicitly Deferred

Feature Area	V1 MVP Scope	Status
Solution Model DSL	Full declarative DSL for solution specification. Too heavy for V1 adoption.	Deferred — V2
Automated Pipeline Orchestration	DAG-based workflow engine, CI/CD orchestration, task automation. Requires ume integration.	Deferred — V2
Full Configuration Management (CM)	Immutable CM-tier, CCB approval flows, build attestations, SLSA provenance. Governance layer.	Deferred — V2
Advanced Testbed System	Structured test environments, automated QA gates, SAST/DAST, benchmarking pipeline.	Deferred — V2
kogi Integration	Portfolio optimization, performance scoring, resource allocation signals from kogi.	Deferred — V3
ume Integration	Organization execution layer, OrgExec task assignment, transformation workflows.	Deferred — V3
Observatory System (advanced)	Full analytics engine, anomaly detection, trend analysis, predictive insights.	Deferred — V2/V3
Supply Chain & Vendor Management	Vendor registry, logistics, physical artifact tracking, inventory management.	Deferred — V3+
IP Management System	Patents, copyrights, licensing, trademark records, contract management.	Deferred — V3+
Solution CPQ	Configure-Price-Quote for market-facing solution offerings.	Deferred — V3+
Energy / Network Resource Mgmt	Power consumption tracking, carbon footprint, bandwidth budgeting.	Deferred — V3+

Tile Recommendation Engine (full)	Pattern-based tile suggestions, system gap analysis, ML-driven surfacing.	Deferred — V2
Multi-Factory Hierarchy	Enterprise, Domain, Team factory nesting with inheritance and governance.	Deferred — V2
Physical Goods Support	Non-software solution types (CPG, agricultural, manufacturing).	Deferred — V3+

6. Core V1 Features — Detailed Specification

6.1 First-Run Experience — The Aha Moment

The first-run experience is the most important product moment in V1. Within 60–90 seconds of connecting a tool, the user must feel: "This understands everything I'm building." The experience must require zero manual input and produce immediate visual payoff.

First-Run Flow

1. Step 0 — Empty State: Minimal workspace. Clear CTA: "Connect your tools to see your system." Buttons: Connect GitHub / Connect Vercel / Connect AWS.
2. Step 1 — Connect GitHub (Primary): OAuth flow → return to Qala. Animated scanning state: "Mapping your system... discovering repositories... identifying services... linking deployments."
3. Step 2 — System Graph Reveal: Fade transition into the auto-generated System Graph. Nodes = services/repos. Edges = dependencies. Layers = deployment environments. No user input required.
4. Step 3 — System Summary Overlay: Card appears: "Here's your system — X services detected, Y environments connected, Z dependencies mapped." Single CTA: "Explore your system."
5. Step 4 — Guided Insights: Subtle highlights surface the first three actionable insights (e.g., "This service has no active deployment", "Changes here affect 3 other services", "This service is only in dev").
6. Step 5 — Next Action Prompt: Soft panel suggests: Connect another tool / Define your solution structure / Enable notifications. No pressure — purely optional guidance.

Design Principles — First-Run

Zero setup thinking — no DSL, no configuration, no forms, no required input
Immediate visual payoff — graph appears in <15 seconds after GitHub OAuth
Uses existing tools — reinforces "I don't have to migrate anything"
Progressive reveal — simple first, depth unlocks naturally
Must feel mostly right — doesn't need to be perfect; must feel accurate enough to trust

Failure Modes to Avoid

- Asking user to manually define system structure — kills the magic
- Showing an empty dashboard after connection — destroys first impression
- Mapping repos incorrectly to the point of confusion — breaks trust immediately
- Too much complexity surfaced upfront — causes cognitive overload and bounce

6.2 Tool Integration Layer

The Integration Layer is a first-class system — not an afterthought. Each integration adapter must ingest state, trigger on change events, and normalize outputs into the unified Qala solution model.

Tool	Auth Method	Ingested Signals	Normalized To
GitHub	OAuth App	Repos, branches, package.json, Dockerfiles, CI config, commits, PRs, workflows	Solution Components, dependency edges, code change events
Vercel	OAuth / API Token	Deployments, project names, environment URLs, build logs, domain mappings	Environment state, deployment status, endpoint mapping
AWS	IAM Role / API Key	ECS/Lambda services, RDS instances, S3 buckets, CloudFormation stacks, ALBs	Environment topology, infrastructure components, deployment targets

Integration Architecture Principles

- **Adapter Pattern:** Each tool has a self-contained adapter that handles auth, polling/webhook ingestion, and data normalization. Adapters are isolated — adding a new integration does not affect others.
- **Event-Driven Updates:** GitHub webhooks trigger real-time graph updates. Polling serves as fallback for tools without webhook support.
- **Normalization Layer:** All adapter outputs are mapped to the canonical Qala data model. No raw tool data leaks into the system graph or solution state.
- **Graceful Degradation:** If an integration goes offline, the system shows cached state with staleness indicators rather than failing.

6.3 System Graph

The System Graph is the primary visual and navigational interface in V1. It is the core differentiator — the feature that delivers the aha moment and proves the thesis.

Graph Structure

- **Nodes:** Services, repositories, and components — each node carries name, type, status, version, and environment assignment.
- **Edges:** Dependencies and relationships — each edge carries direction, type (hard dependency, soft dependency, data flow), and impact weight.
- **Layers:** Code layer (repos/components), runtime layer (deployed services), and environment layer (dev, staging, production).

Graph Capabilities — V1

- Auto-generated from tool integrations — no manual definition required
- Interactive — click nodes to see details; zoom, pan, and filter

- Right panel on node click: service name, type, source, environment, last deploy, status, dependencies
- Dependency impact awareness: "Changes here affect X other systems"
- Environment mismatch detection: "This service is only deployed in dev"
- Missing deployment highlighting: "This service has no active deployment"
- Real-time updates when deployment status changes or new commits land

Graph is NOT

- A manual architecture diagramming tool — it is derived state, not user-authored content
- A replacement for Lucidchart or Miro — it is a live, executable system view

6.4 Solution Registry

The Solution Registry is the authoritative catalog of all solutions being built within a workspace. V1 keeps this lightweight — structured enough to be useful, minimal enough to avoid friction.

Solution Record — V1 Fields

Field	Type	V1 Support	Notes
Unique ID	UUID v4	Auto-generated	Immutable after creation
Name	String	Required	Unique within workspace
Version	Semver	Manual input	e.g., 0.1.0
Maturity	Enum	Manual promotion	SANDBOX | DEV | TEST (CM deferred)
Solution Type	Enum	Required	Application | Service | Platform | System | Product
Owner	User ref	Required	Primary owner assignment
Value Proposition	Text	Optional	One-paragraph statement of what this solves
Status	Enum	Auto + Manual	Active | Deprecated | Archived
Tags	String[]	Optional	Freeform labels for search and filter
Created / Updated At	Timestamp	Auto	ISO 8601
SDE Reference	Repo refs	Auto-linked from GitHub	Linked source repositories

Solution Maturity Lifecycle — V1

V1 supports three maturity stages. Promotion is manual. Gate enforcement is awareness-only — the system surfaces what is missing but does not block promotion in V1.

Stage	Description	Gate Awareness (V1)
SANDBOX	Exploratory, unconstrained. No stability guarantees.	None — open creation
DEV	Active feature development. Connected to a repository.	Solution record created; owner assigned; repo linked
TEST	Feature-complete candidate. Testing in progress.	Components defined; basic deployment connected

6.5 Dashboard — Tile System

The Dashboard is the operational control surface of V1. It is not just a display layer — it is the interface through which users interpret system state and take action. V1 builds the foundation of the tile system with just enough capability to be genuinely useful.

V1 Tile Types

Tile Type	Purpose	V1 Data Source	Interactions
Status Tile	Service health, deployment state for a solution or component	Tool integrations (GitHub, Vercel, AWS)	Click to open solution detail
Metric Tile	Single numeric value (deploys this week, open PRs, uptime)	Tool integrations, computed aggregates	Click to drill into source
Alert Tile	Active failure or anomaly requiring attention	Event stream from integrations	Click to acknowledge / investigate
Relationship Tile	Snippet of dependency graph for a specific solution	System graph engine	Click to open full graph view
Action Tile	Trigger an action: open in GitHub, view deployment, open logs	Deep links to integrated tools	Click to execute action

V1 Tile Engine — What Is Built

- Tile object model — stateful, typed tile records with id, type, source, data, state, priority, group, presentation metadata
- Tile lifecycle management — create, update, refresh, destroy
- Basic grouping — user-defined sections (e.g., "System Health", "Deployments", "AI Pipelines")
- Drag-and-drop layout — manual tile arrangement within groups
- Real-time updates — WebSocket connection; tiles update on deployment events and status changes
- Basic relevance sorting — priority = system_impact × 0.4 + recency × 0.2 + user_interest × 0.2 + risk_level × 0.2

- Manual pinning — pin critical tiles to always-visible positions
- Tile search and filter — find tiles by solution, status, or type

V1 Tile Engine — What Is Deferred

- Full recommendation engine — ML-based pattern recognition and gap analysis
- Advanced adjudication logic — complex multi-signal layout optimization
- Confidence scoring and risk modeling — data freshness and source reliability scoring

6.6 Solution Workspace

The Solution Workspace is the detail view for a single solution. It gives owners a structured operational home for everything related to their solution. V1 keeps this focused on the essentials.

V1 Workspace Sections

- **Overview:** Solution metadata, value proposition, maturity badge, owner, version, tags.
- **Components:** List of linked components (repos/services) with status indicators and last-updated timestamps.
- **Environments:** Deployment state per environment (dev, staging, prod) with last deploy time, status, and deployment URL.
- **System Graph Snippet:** Embedded mini-graph showing this solution's node and immediate dependencies.
- **Solution Book (lite):** Collapsible charter panel with vision, mission, brief, goals, notes. Document attachment (PDF, MD).
- **Activity Feed:** Chronological log of deployment events, status changes, and user actions for this solution.

7. Technical Architecture — V1

The V1 architecture is designed for speed-to-market and correctness over completeness. It establishes the core patterns that the full platform will scale on, without over-engineering for capabilities that are explicitly deferred.

7.1 Architecture Overview

Layer	Technology	Responsibility
Frontend	React + TypeScript, Tailwind CSS	Workspace UI, System Graph (D3.js / Cytoscape), Dashboard tile renderer, WebSocket client
API Gateway	GraphQL (Apollo) + REST for webhooks	Single entry point for all client queries and mutations; webhook ingestion endpoint
Solution State Engine	Node.js / TypeScript service	Central data model: solutions, components, dependencies, environments, maturity state
Integration Layer	Adapter services (one per tool)	GitHub / Vercel / AWS adapters; OAuth flows; data ingestion and normalization
Graph Engine	Graph DB (Neo4j or PostgreSQL w/ pgvector)	Dependency graph storage, traversal, impact analysis
Tile Engine	Node.js service + Redis	Tile state management, relevance scoring, event routing, WebSocket push
Event Bus	Kafka (or Redis Streams in V1)	Async event routing: integration events → state engine → tile engine → UI
Primary Database	PostgreSQL	Solution records, user/workspace data, tile definitions, integration configs
Cache	Redis	Tile state cache, session management, rate limiting
Auth	Auth0 / Clerk	OAuth flows (GitHub, Google), JWT session management, RBAC
Infrastructure	AWS (ECS / Fargate)	Containerized services; RDS for Postgres; ElastiCache for Redis; MSK for Kafka

7.2 Data Flow — System Graph Generation

The following describes the core data flow that produces the aha moment: connect GitHub → see your system.

7. User authenticates with GitHub via OAuth. GitHub adapter stores access token securely.
8. GitHub adapter scans all accessible repositories. For each repo: extracts name, language, package dependencies (package.json, go.mod, requirements.txt, Cargo.toml), Dockerfile presence, and CI/CD config.

9. Adapter normalizes each repo into a Solution Component record and posts to the Solution State Engine via internal API.
10. Solution State Engine infers dependency edges from package dependency data and creates/updates the graph.
11. If Vercel or AWS are also connected, their adapters contribute environment nodes and deployment state edges to the graph.
12. Graph Engine stores the complete solution graph. System Graph API serves the computed graph to the frontend.
13. Frontend renders the graph in <15 seconds total from OAuth callback. WebSocket channel established for real-time updates.

7.3 Performance Targets

Metric	V1 Target	Rationale
First graph render after GitHub OAuth	< 15 seconds	Critical for aha moment — any longer breaks magic
Tile load time on dashboard open	< 200ms	Dashboard must feel instant
WebSocket event-to-tile update latency	< 2 seconds	Real-time feel for deployment status changes
System graph with 100 nodes	< 3 seconds render	Typical indie/small-team system size
Solution Registry search results	< 500ms	Search must feel snappy

7.4 Security Model — V1

- All OAuth tokens encrypted at rest (AES-256); never logged or exposed in API responses
- Workspace-level isolation — users can only access solutions within their workspace
- RBAC: Owner (full access), Member (read/write), Viewer (read-only)
- Webhook validation — all incoming GitHub/Vercel webhooks verified by signature before processing
- Audit log — all solution state mutations recorded with user, timestamp, and change delta

8. Success Metrics — V1

V1 success is defined by two categories: aha moment validation and sustained engagement. If users connect tools, see their system graph, and return in subsequent sessions — V1 has succeeded.

8.1 Activation Metrics

Metric	Target	Definition
Tool connection rate	> 80% of signups	Users who connect at least one tool within 24 hours of signup
System graph generated	> 75% of connected users	Users who see a populated system graph within the first session
Aha moment time	< 90 seconds	Median time from first tool connection to first graph view
First session duration	> 8 minutes	Users spending meaningful time exploring their system

8.2 Engagement Metrics (D7 / D30)

Metric	Target	Definition
D7 Retention	> 40%	Users returning at least once in the 7 days after signup
D30 Retention	> 25%	Users returning at least once in the 30 days after signup
Solutions created	> 2 per active user	Users who define at least 2 solutions in their registry
Tool integrations per workspace	> 2	Average number of tools connected per active workspace
Dashboard tiles configured	> 5 per active user	Users who have set up a meaningful dashboard

8.3 Qualitative Signals

- User interviews: "I understood my system for the first time" within the first session
- Word-of-mouth: organic sharing of system graph screenshots in dev communities
- Tool request volume: users actively requesting integrations for tools not yet supported
- Feedback quality: requests for deeper features (not complaints about basics)

9. Phased Roadmap

Phase 1 — V1 MVP (Months 1–4)

Deliver the core thesis. Everything needed to go from "connect tools" to "see your system" with a live, interactive dashboard.

Deliverable	Description	Priority
Auth + Workspace Setup	Email auth, OAuth sign-in, workspace creation, factory initialization	Must Have
GitHub Integration Adapter	OAuth, repo scan, dependency inference, webhook ingestion	Must Have
System Graph (V1)	Auto-generated graph, interactive nodes/edges, right-panel detail	Must Have
Solution Registry (lite)	CRUD solutions, metadata, maturity lifecycle (SANDBOX/DEV/TEST)	Must Have
Solution Workspace (lite)	Solution detail page, components, environments, activity feed	Must Have
Tile Dashboard (V1)	Tile object model, 5 tile types, grouping, layout, real-time updates	Must Have
Vercel Integration Adapter	OAuth, deployment ingestion, environment state sync	Should Have
AWS Integration Adapter	IAM auth, service discovery, deployment state	Should Have
Deployment State Tracking	Per-solution deployment status across environments	Must Have
First-Run Experience	Guided connect flow, animated graph reveal, insight highlights	Must Have
Notifications (basic)	Email + in-app alerts for deployment failures and status changes	Should Have
Solution Book (lite)	Charter fields, notes, document attachment	Could Have

Phase 2 — V2 (Months 5–9)

Deepen the model. Introduce workflow orchestration, advanced dashboard intelligence, and the first governance layer.

- Work System integration — lightweight task management linked to solution components
- Pipeline State Tracking — CI/CD pipeline runs visible in system graph and dashboard tiles
- Multi-tool integrations — Linear, Datadog, Slack, GitLab, Railway
- Tile Recommendation Engine — pattern-based tile suggestions and system gap analysis
- Multi-Factory Hierarchy — team and organization factory nesting

- Advanced Solution Modeling — component interface contracts, dependency locking, version pinning
- Workflow Orchestration (lite) — simple DAG-based workflows for release and deployment processes
- Solution Collaboration — multi-user co-editing, comments, and change attribution

Phase 3 — V3 (Months 10–18)

Introduce the full platform: governance, optimization loop, and advanced automation.

- Solution Model DSL — declarative solution specification language; gradual evolution from structured forms
- Full Configuration Management — CM-tier maturity, CCB approval workflows, build attestation, SLSA provenance
- kogi Integration — portfolio-level optimization signals fed into dashboard and solution recommendations
- ume Integration — organization execution layer, OrgExec task routing, transformation workflows
- Observatory System — full analytics engine, anomaly detection, trend analysis, predictive insights
- Advanced Testbed System — automated QA gates, security scanning, performance benchmarking pipelines
- Enterprise Factory Hierarchy — full governance inheritance, policy management, compliance reporting
- Solution Marketplace — publish and consume solution templates, reference architectures, and factory blueprints

Phase 4 — V4 (Month 18+)

Full solution factory system. AI-driven orchestration. Cross-domain expansion.

- AI-driven solution generation — AI agents co-authoring solution models and blueprints
- Full kogi optimization loop — closed-loop define → execute → measure → optimize cycle
- Physical goods and service solution types — non-software solution management
- Supply chain and logistics management — vendor registry, inventory, distribution channels
- IP Management System — patents, copyrights, licensing, contracts

10. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation Strategy
Aha moment fails — graph is inaccurate or confusing	Medium	Critical	Invest heavily in inference quality; show confidence indicators; let users manually correct nodes; accuracy target is "feels mostly right" not perfect
Over-complexity creeps into V1 scope	High	High	Weekly scope review; any feature not directly serving the aha moment or first 7-day retention is deferred without exception
Tool API limitations block integration depth	Medium	Medium	Start with highest-signal endpoints; design adapters to be progressively enhanced; fallback to manual component creation
Low adoption — market not ready for system-level thinking	Medium	High	Target only founders who personally feel the pain; lead with visual demo not concept explanation; grow via word-of-mouth from highly satisfied early users
Dashboard is too complex for new users	Medium	High	V1 dashboard starts pre-populated with inferred tiles; user can simplify; complexity is progressive, not forced
GitHub-only integrations limit value for non-GitHub users	Low	Medium	Phase 1 milestone: GitLab adapter in V2; communicate roadmap clearly; GitHub covers >70% of target persona
Dependency inference is wrong for monorepos or unusual structures	High	Medium	Allow manual override on any inferred relationship; flag low-confidence inferences with explanation

11. Positioning & Go-To-Market

11.1 Core Positioning

> ***"Qala does not replace your tools — it makes them finally work together as a system."***

Positioning Element	Statement
Headline	Orchestrate your entire system — without replacing your stack
Subheadline	Connect your tools, structure your solutions, and operate everything as one system.
One-liner	Qala is a system orchestration layer for solutions — not a replacement for your tools.
Key benefit 1	Works with your existing tools — GitHub, Vercel, AWS — no migration required
Key benefit 2	Auto-generates a live system graph in under 90 seconds from your first tool connection
Key benefit 3	Tracks development, deployment, and dependency state in one unified model
Key benefit 4	Gives you a real-time decision surface through an intelligent, composable dashboard

11.2 Competitive Positioning

Competitor	What They Do	Their Limitation	Qala's Position
GitHub	Code hosting and basic CI/CD	Code-only; no system model; no cross-repo orchestration	System orchestration layer above code hosting
Backstage	Static service catalog	Document-based; not live or executable; requires heavy setup	Live, executable, state-driven version of the same concept
Notion	Documentation and wikis	No execution, no live data, no system awareness	Operational system model, not just documentation
Linear	Task and issue tracking	Task-only; no connection to deployment state or system topology	System context for work — not just work management
Datadog	Observability and monitoring	Observability of running systems; no solution model or composition	System definition layer that feeds into observability

11.3 Go-To-Market Strategy

Phase 1 — Founder-Led Adoption (V1)

- Target: Indie hackers, AI builders, founders of multi-system platforms — people who personally feel the pain every day.
- Channel: Developer communities (X/Twitter, Hacker News, Discord), personal network outreach, targeted DMs to builders publicly expressing system complexity pain.
- Motion: Demo-first. Never explain the concept — show the graph. "Here is what your GitHub repos look like as a system."
- Conversion trigger: Free tier for individuals; personal factory with up to 5 solutions and 2 integrations.
- Retention driver: The aha moment generates daily habitual use — builders check system state every morning the way they check email.

Phase 2 — Team Expansion (V2)

- Target: Pre-PMF teams of 3–15 engineers adopting as shared system-of-record.
- Motion: Bottom-up land via individual power users; expand through team invite. Collaboration features make individual value a team necessity.
- Pricing: Team tier unlocks multi-user factory, collaboration, and additional integrations.

Phase 3 — Platform Adoption (V3+)

- Target: Internal platform teams at growth-stage and enterprise organizations.
- Motion: Top-down sale to engineering leadership after bottom-up adoption demonstrates internal ROI.
- Value: Governance, compliance traceability, audit trails, and standardization across teams.

12. Conclusion

Qala is a paradigm-level shift in how software systems are understood and operated. The full vision — a deterministic, state-driven solution production system spanning every solution type from software applications to physical goods — is a multi-decade architectural project.

V1 is not the full vision. It is the proof. One, high-value, non-negotiable thesis: connect your tools and see your entire system in 90 seconds — without changing anything about how you work.

If V1 succeeds in delivering that thesis with unmistakable clarity, the platform earns the right to grow: deeper integrations, richer governance, workflow orchestration, and eventually the full closed-loop optimization cycle with kogi and ume.

V1 Success Definition
Users see their system graph within 90 seconds of connecting GitHub.
Users return to Qala the next day, the day after that, and the day after that.
Users say: "I finally understand everything I'm building — as one system."
That is V1. Everything else is V2, V3, and beyond.

> ***"The layer where systems are defined, understood, and operated — not just built."***

— Qala V1 Product Proposal, April 2026